

Lossless Source Coding

Prof. Bikash Kumar Dey

IIT Bombay

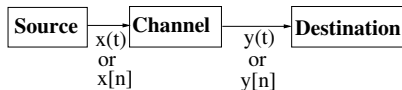
January 19, 2026

Outline

- 1 Introduction: Source Coding
- 2 Lossless Source Coding
- 3 Huffman Code
- 4 Kraft Inequality and Optimal Codes
- 5 Optimal Codes / Codelength
- 6 Lempel-Ziv Code
- 7 Shannon-Fano-Elias Code
- 8 Arithmetic Code
- 9 Arithmetic Code: Part 1
- 10 Arithmetic Code: part 2

Source Coding

Objective: To represent source at a lower rate.



A Typical Communication System

Why source Coding?

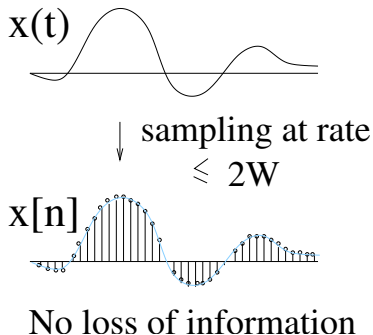
$x(t)/x[n]$ may have **redundancy** or undesired/unimportant extra information

Telephone quality speech signal can be compressed to as little as 1-2Kbps.

Source Coding

$x(t)/x[n]$ may have **redundancy** or undesired/unimportant extra information

Example 1: A band-limited continuous time signal, bandwidth W

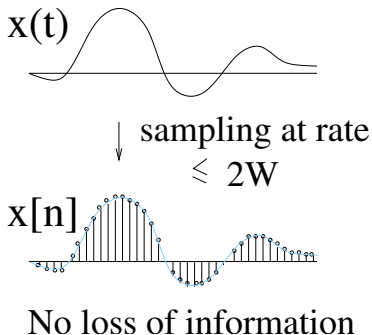


⇒ intermediate values redundant!

Source Coding

$x(t)/x[n]$ may have **redundancy** or undesired/unimportant extra information

Example 1: A band-limited continuous time signal, bandwidth W



$\Rightarrow$ **intermediate values redundant!**

Source Coding

- The channel may not have the “capacity” to communicate at the source information rate $\implies$ need to represent source at a lower rate with *some loss of information*

Example: A real valued sample has infinite information. It needs to be **quantized** before transmission through a DMC

- Non-uniform distribution of the source may allow efficient representation of the signal at a lower rate

Example: For binary transmission of English text, frequent letters may be represented by shorter bit sequences, e.g. **Morse Code**.

Telegraph and Morse code

International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

A • —
B — • • •
C — • — •
D — • •
E •
F • • — •
G — — •
H • • • •
I • •
J • — — —
K — • —
L — • • •
M — —
N — •
O — — —
P • — — •
Q — — • —
R — • •
S • • •
T —

U • • —
V • • • —
W • — —
X — • • —
Y — • — —
Z — — • •

1 • — — — —
2 • • — — —
3 • • • — —
4 • • • • —
5 • • • • •
6 — • • • •
7 — — • • •
8 — — — • •
9 — — — •
0 — — — — •



Types of Source Coding

- **Lossless**

- ▶ no loss of information
- ▶ used by various compression schemes for file storage (e.g. **compress** in Linux)

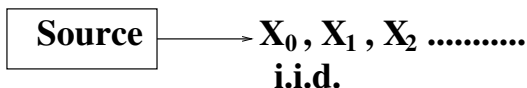
- **Lossy**

- ▶ with loss of information, e.g. quantization, vector quantization, sub-band coding
- ▶ used by standards like **JPEG, MPEG etc.**

Lossless Source Coding for Discrete Memoryless Sources

Huffman Code

Discrete Memoryless source



$$X_i \in \{x_0, x_1, \dots, x_{M-1}\}$$

$$P\{X_i = x_0\}, P\{X_i = x_1\}, \dots, P\{X_i = x_{M-1}\} - \text{known}$$

Example: An analog source sampled and quantized

If $M \leq 2^b$, then we can represent the symbols by binary fixed length codes as:

Symbol x_i	Codes $C(x_i)$	Length l_i	Probability P_i
x_0	0 ... 000	$l_0 = b$	P_0
x_1	0 ... 001	$l_1 = b$	P_1
x_2	0 ... 010	$l_2 = b$	P_2
$\vdots$	$\vdots$	$\vdots$	$\vdots$

This will require b -bits per symbol.

Something better possible!

Example:

Symbol:	x_0	x_1	x_2	x_3
Probability:	0.5	0.25	0.125	0.125
Fixed length:	00	01	10	11
Variable length:	0	10	110	111

Average length for fixed length code = 2,

Average length for variable length code = 1.75

Entropy $H(x) = 1.75$

Desired properties of source codes

- **Non-singular** : $x_i \neq x_j \Rightarrow C(x_i) \neq C(x_j)$
Sequence of codewords may not be uniquely decodable

Example: $\{0, 010, 01, 10\}$

- **Uniquely Decodable**: For any two finite sequences

$$x_1 x_2 \dots x_n \neq x'_1 x'_2 \dots x'_m$$

$$\Rightarrow C(x_1)C(x_2)\dots C(x_n) \neq C(x'_1)C(x'_2)\dots C(x'_m)$$

But the decoder may need to observe future bits

Desired properties of source codes

- **Non-singular** : $x_i \neq x_j \Rightarrow C(x_i) \neq C(x_j)$
Sequence of codewords may not be uniquely decodable

Example: $\{0, 010, 01, 10\}$

- **Uniquely Decodable**: For any two finite sequences

$$x_1 x_2 \dots x_n \neq x'_1 x'_2 \dots x'_m$$

$$\Rightarrow C(x_1)C(x_2)\dots C(x_n) \neq C(x'_1)C(x'_2)\dots C(x'_m)$$

But the decoder may need to observe future bits

Desired properties of source codes

- Prefix code or Instantaneous code: no codeword is a prefix of any other codeword
-codes can be decoded instantaneously!

Desired properties of source codes

- Prefix code or Instantaneous code: no codeword is a prefix of any other codeword
-codes can be decoded instantaneously!

Example : Classes of codes

X	Singular	Non-singular but not UD	UD, but not Prefix	prefix
1	0	0	10	0
2	0	010	00	10
3	0	01	11	110
4	0	10	110	111

Huffman code

Example

X	Codeword	Probability
1		0.20
2		0.15
3		0.25
4		0.25
5		0.15

Example

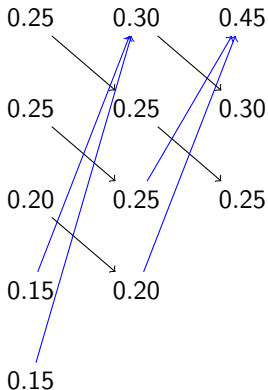
X	Codeword	Probability
3		0.25
4		0.25
1		0.20
2		0.15
5		0.15

Example

X	Codeword	Probability
3	0 1	0.25 0.30
4		0.25 0.25
1		0.20 0.25
2		0.15 0.20
5		0.15

Example

X	Codeword	Probability
3	0	0.25 0.30 0.45
4		0.25 0.25 0.30
1	1	0.20 0.25 0.25
2	0	0.15 0.20
5	1	0.15



Example

X	Codeword	Probability
3	1	0.25 0.30 0.45 0.55
4	0	0.25 0.25 0.30 0.45
1	1	0.20 0.25 0.25
2	0 0	0.15 0.20
5	0 1	0.15

Example

X	Codeword	Probability
3	0 1	0.25 0.30 0.45 0.55 → 1
4	1 0	0.25 0.25 0.30 0.45
1	1 1	0.20 0.25 0.25
2	0 0 0	0.15 0.20
5	0 0 1	0.15

Huffman code

Code Design

- Arrange the alphabet in decreasing order of their probability
- Assign 0 and 1 as the last bit of the last two symbols
- Combine the last two symbols as one and assign the sum of their probabilities to the new symbol
- Repeat this process on the new set of symbol again and again. While assigning a bit to a derived symbol, the bit is added in the beginning of the codewords of all the contributing symbols.

Properties and comments

- Huffman code is a prefix code.
- Huffman code is optimum - no code with smaller average length than Huffman code for any random variable.
- Non-binary (q -ary) code: Add 0 probability symbols if necessary so we have $k(q - 1) + 1$ number of symbols. Combine q symbols at each step.

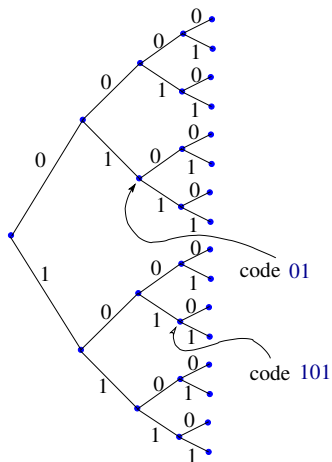
Kraft Inequality

Kraft Inequality:

A prefix code with codeword lengths $l_1, l_2, l_3, \dots, l_M$ exists if and only if

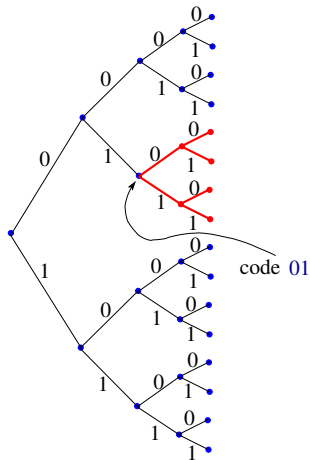
$$\sum_i 2^{-l_i} \leq 1$$

Kraft Inequality: Proof



Necessity:

Kraft Inequality: Proof

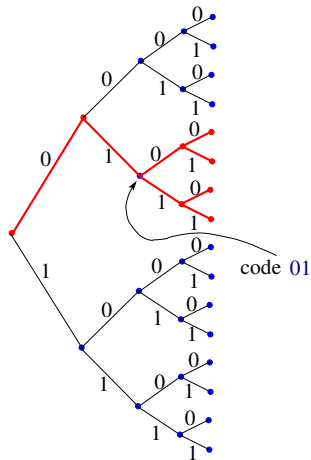


Necessity:

Codewords disqualified by 01:

010,011,0100,0101, 0110,0111

Kraft Inequality: Proof

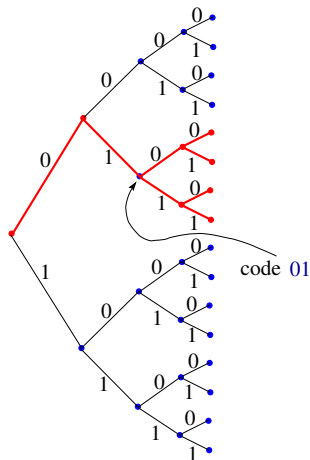


Necessity:

Codewords disqualified by 01:

010,011,0100,0101, 0110,0111,0

Kraft Inequality: Proof



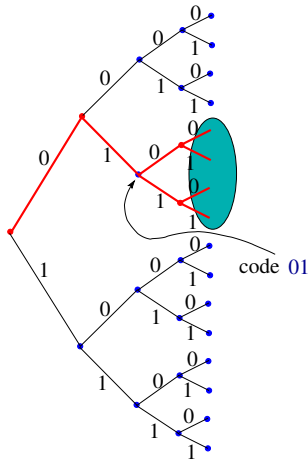
Necessity:

Codewords disqualified by 01:

010,011,0100,0101, 0110,0111,0

Consider codeword $C(x_i)$, length l_i and let $l_{max} = \max_i \{l_i\}$

Kraft Inequality: Proof



Necessity:

Codewords disqualified by 01:

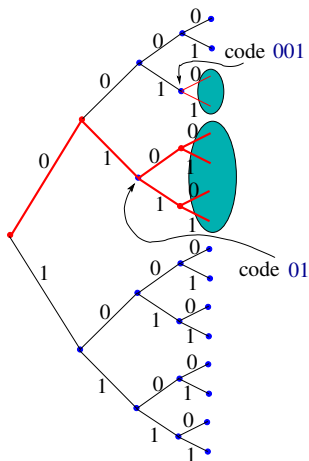
010,011,0100,0101, 0110,0111,0

Consider codeword $C(x_i)$, length l_i and let $l_{max} = \max_i \{l_i\}$

No. of descendants of $C(i)$ at level l_{max} $= 2^{l_{max}-l_i}$

Descendants are disjoint for different i

Kraft Inequality: Proof



Necessity:

Codewords disqualified by 01:

010,011,0100,0101, 0110,0111,0

Consider codeword $C(x_j)$, length l_j and let

$$l_{max} = \max_i \{l_i\}$$

No. of descendants of $C(i)$ at level

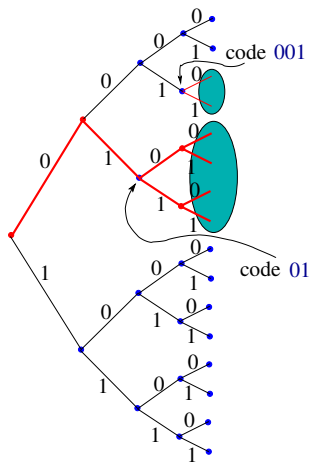
$$l_{max} = 2^{l_{max}} - l_i$$

Descendants are disjoint for different i

Considering all codewords,

$$\sum_i 2^{l_{\max} - l_i} \leq 2^{l_{\max}}$$

Kraft Inequality: Proof



Sufficiency:

$l_1, l_2, \dots, l_M$ - Given in increasing order.

To construct a prefix code

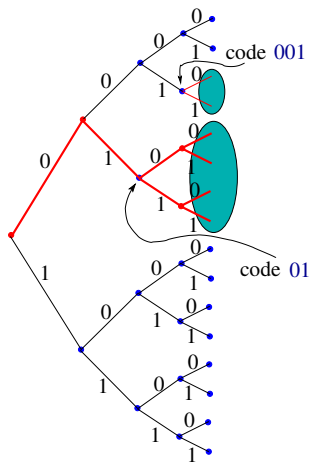
For $i = 1, 2, \dots, M$,

Take the first remaining node (*) at level l_i as $C(x_i)$.

Such a node exists since $\sum_{j < i} 2^{l_j - l_i} < 2^{l_i}$.

(*) A node that is not a descendant of any code chosen so far.

Kraft Inequality: Proof

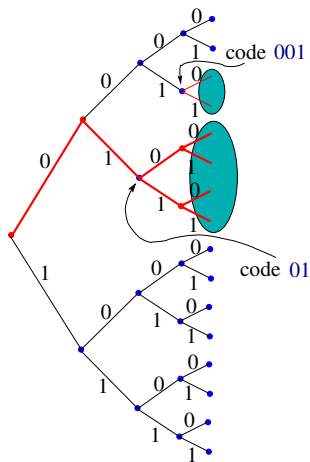


Sufficiency:

$l_1, l_2, \dots, l_M$ - Given in increasing order.

(*) A node that is not a descendant of any code chosen so far.

Kraft Inequality: Proof



Sufficiency:

$l_1, l_2, \dots, l_M$ - Given in increasing order.

To construct a prefix code

For $i = 1, 2, \dots, M$,

Take the first remaining node (*) at level l_i as $C(x_i)$.

Such a node exists since $\sum_{j < i} 2^{l_j - l_j} < 2^{l_i}$.

(*) A node that is not a descendant of any code chosen so far.

Kraft Inequality: Comments

- Kraft inequality also holds for uniquely decodable codes
- UDC with $l_1, l_2, \dots, l_m$ exists $\Rightarrow$ prefix code with $l_1, l_2, \dots, l_m$ exists
- Non-binary codes: For q -ary codes, the condition is

$$\sum_i q^{-l_i} \leq 1$$

Kraft Inequality: Comments

- Kraft inequality also holds for uniquely decodable codes
- UDC with $l_1, l_2, \dots, l_m$ exists $\Rightarrow$ prefix code with $l_1, l_2, \dots, l_m$ exists
- Non-binary codes: For q -ary codes, the condition is

$$\sum_i q^{-l_i} \leq 1$$

Kraft Inequality: Comments

- Kraft inequality also holds for uniquely decodable codes
- UDC with $l_1, l_2, \dots, l_m$ exists $\Rightarrow$ prefix code with $l_1, l_2, \dots, l_m$ exists
- Non-binary codes: For q -ary codes, the condition is

$$\sum_i q^{-l_i} \leq 1$$

Optimal Codes / Codelength

Optimal code

Average length of a code:

$$L(C, X) = \sum_i p_i l_i.$$

A code for an alphabet $x_1, x_2, \dots, x_m$ is optimal if there is no code for the same alphabet with smaller average length.

Denote the average length of an optimal code as L .

Theorem

$$H(x) \leq L < H(x) + 1$$

Optimal code

Average length of a code:

$$L(C, X) = \sum_i p_i l_i.$$

A code for an alphabet $x_1, x_2, \dots, x_m$ is optimal if there is no code for the same alphabet with smaller average length.

Denote the average length of an optimal code as L .

Theorem

$$H(x) \leq L < H(x) + 1$$

Part I: For any uniquely decodable code, the expected length $L(C, X) \geq H(X)$

Define $z = \sum_j 2^{-l_j} \leq 1$ and $q_i := 2^{-l_i}/z$. Hence $l_i = \log(1/q_i) - \log(z)$.

Known inequality: $\sum_i p_i \log\left(\frac{1}{q_i}\right) \geq \sum_i p_i \log\left(\frac{1}{p_i}\right)$.

equality iff $p_i = q_i \forall i$

$$\begin{aligned} L(C, X) &= \sum_i p_i l_i = \sum_i p_i \log\left(\frac{1}{q_i}\right) - \log(z) \\ &\geq \sum_i p_i \log\left(\frac{1}{p_i}\right) - \log(z) \geq H(x) \end{aligned}$$

Part I: For any uniquely decodable code, the expected length $L(C, X) \geq H(X)$

Define $z = \sum_j 2^{-l_j} \leq 1$ and $q_i := 2^{-l_i}/z$. Hence $l_i = \log(1/q_i) - \log(z)$.

Known inequality: $\sum_i p_i \log\left(\frac{1}{q_i}\right) \geq \sum_i p_i \log\left(\frac{1}{p_i}\right)$.

equality iff $p_i = q_i \forall i$

$$\begin{aligned} L(C, X) &= \sum_i p_i l_i = \sum_i p_i \log\left(\frac{1}{q_i}\right) - \log(z) \\ &\geq \sum_i p_i \log\left(\frac{1}{p_i}\right) - \log(z) \geq H(x) \end{aligned}$$

Part I: For any uniquely decodable code, the expected length $L(C, X) \geq H(X)$

Define $z = \sum_j 2^{-l_j} \leq 1$ and $q_i := 2^{-l_i}/z$. Hence $l_i = \log(1/q_i) - \log(z)$.

Known inequality: $\sum_i p_i \log\left(\frac{1}{q_i}\right) \geq \sum_i p_i \log\left(\frac{1}{p_i}\right)$.

equality iff $p_i = q_i \forall i$

$$\begin{aligned} L(C, X) &= \sum_i p_i l_i = \sum_i p_i \log\left(\frac{1}{q_i}\right) - \log(z) \\ &\geq \sum_i p_i \log\left(\frac{1}{p_i}\right) - \log(z) \geq H(x) \end{aligned}$$

Part I: For any uniquely decodable code, the expected length $L(C, X) \geq H(X)$

Equality $L(C, X) = H(X)$ is satisfied iff

- ① $\log(z) = 0 \Rightarrow \sum_j 2^{-l_j} = 1$, i.e., equality in Kraft inequality.
- ② $l_i = \log\left(\frac{1}{p_i}\right)$

Part II: There exists code with $L(C, X) < H(X) + 1$

For all i , define $l_i = \lceil \log_2(\frac{1}{p_i}) \rceil$

Kraft inequality satisfied by l_i s:

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2(\frac{1}{p_i}) \rceil} \leq \sum_i p_i = 1$$

$\Rightarrow$ There is a prefix code with these l_i s.

$$\begin{aligned} L(C, X) &= \sum_i p_i \lceil \log_2(\frac{1}{p_i}) \rceil \\ &< \sum_i p_i \left(\log_2\left(\frac{1}{p_i}\right) + 1 \right) \\ &= H(X) + 1 \end{aligned}$$

Part II: There exists code with $L(C, X) < H(X) + 1$

For all i , define $l_i = \lceil \log_2(\frac{1}{p_i}) \rceil$

Kraft inequality satisfied by l_i s:

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2(\frac{1}{p_i}) \rceil} \leq \sum_i p_i = 1$$

$\Rightarrow$ There is a prefix code with these l_i s.

$$\begin{aligned} L(C, X) &= \sum_i p_i \lceil \log_2(\frac{1}{p_i}) \rceil \\ &< \sum_i p_i \left(\log_2\left(\frac{1}{p_i}\right) + 1 \right) \\ &= H(X) + 1 \end{aligned}$$

Part II: There exists code with $L(C, X) < H(X) + 1$

For all i , define $l_i = \lceil \log_2(\frac{1}{p_i}) \rceil$

Kraft inequality satisfied by l_i s:

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2(\frac{1}{p_i}) \rceil} \leq \sum_i p_i = 1$$

$\Rightarrow$ There is a prefix code with these l_i s.

$$\begin{aligned} L(C, X) &= \sum_i p_i \lceil \log_2(\frac{1}{p_i}) \rceil \\ &< \sum_i p_i \left(\log_2\left(\frac{1}{p_i}\right) + 1 \right) \\ &= H(X) + 1 \end{aligned}$$

Part II: There exists code with $L(C, X) < H(X) + 1$

For all i , define $l_i = \lceil \log_2(\frac{1}{p_i}) \rceil$

Kraft inequality satisfied by l_i s:

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2(\frac{1}{p_i}) \rceil} \leq \sum_i p_i = 1$$

$\Rightarrow$ There is a prefix code with these l_i s.

$$\begin{aligned} L(C, X) &= \sum_i p_i \lceil \log_2(\frac{1}{p_i}) \rceil \\ &< \sum_i p_i \left(\log_2\left(\frac{1}{p_i}\right) + 1 \right) \\ &= H(X) + 1 \end{aligned}$$

Part II: There exists code with $L(C, X) < H(X) + 1$

For all i , define $l_i = \lceil \log_2(\frac{1}{p_i}) \rceil$

Kraft inequality satisfied by l_i s:

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2(\frac{1}{p_i}) \rceil} \leq \sum_i p_i = 1$$

$\Rightarrow$ There is a prefix code with these l_i s.

$$\begin{aligned} L(C, X) &= \sum_i p_i \lceil \log_2(\frac{1}{p_i}) \rceil \\ &< \sum_i p_i \left(\log_2\left(\frac{1}{p_i}\right) + 1 \right) \\ &= H(X) + 1 \end{aligned}$$

Mismatched code

Suppose a code is designed to be optimal for a distribution q , while the actual distribution of the source is p . Then what is the penalty in average length?

$$\begin{aligned}l_i &= \log_2 \left(\frac{1}{q_i} \right) . \\ \text{Average length} &= \sum_i p_i l_i \\ &= \sum_i p_i \log_2 \left(\frac{1}{q_i} \right) \\ &= \sum_i p_i \log_2 \left(\frac{p_i}{q_i} \right) - \sum_i p_i \log_2 p_i \\ &= D(p||q) + H(X)\end{aligned}$$

$$\text{Penalty} = D(p||q).$$

Need to combine many source symbols for source coding

symbols	probability	best symbol-wise code
1	0.9	1
0	0.1	0

Average length = 1 bit

$$\begin{aligned}\text{Entropy} &= -(0.9 \log_2(0.9) + 0.1 \log_2(0.1)) \\ &= 0.469 \text{ bits}\end{aligned}$$

Symbol-wise coding can assure only $L < H(X) + 1$.

Source coding theorem says: $L \rightarrow H(X)$

How $L \rightarrow H(X)$ can be achieved for any DMS:

- Combine n samples $(X_t, X_{t+1}, \dots, X_{t+n-1})$ and consider it as a single source symbol
- Entropy of $(X_t, X_{t+1}, \dots, X_{t+n-1})$ is $nH(x)$
- The optimal code length L_n for $(X_t, X_{t+1}, \dots, X_{t+n-1})$ satisfies
$$nH(x) \leq L_n < nH(x) + 1$$
- The average code length $L = \frac{L_n}{n}$ per actual source symbol satisfies
$$H(x) \leq L < H(x) + \frac{1}{n}$$
- By increasing n , we can have L as close to $H(x)$ as desired.

Huffman Code Revisited

Properties

- Huffman code is a prefix code.
- Huffman code is optimum - no code better than Huffman code for any alphabet.
- It can be used to encode optimally with $L \rightarrow H(X)$ as $n \rightarrow \infty$.

Huffman Code Revisited

Disadvantages

- Need to know/ calculate the symbol probabilities explicitly $\Rightarrow$ to combine n symbols of an M -ary source, need to compute M^n probabilities.
- Can not apply for sources with unknown statistics.

Lempel-Ziv Coding

LZ Code

- No explicit knowledge of the source statistics required. [Universal](#).
- Asymptotically optimum for memoryless sources.
- Compress in Linux uses *LZ* algorithm.

LZ Code: How!

- ① Maintain a list of substrings appeared before.
- ② Store the output string from the source in a buffer.
- ③ Check if the present string at the buffer is their in the list.
 - ▶ If yes, then wait for one more symbol to come into the buffer and then go back to step 3.
 - ▶ If no, then
 - ★ find the substring (buffer string excluding the last symbol) in the list and transmit its index.
 - ★ Transmit the last symbol.
 - ★ Empty the buffer.

Encoding: Example

Input:

0

□

Output:

Encoding: Example

Input: 1

0	1
---	---

☐	1
--------------------------	---

Output: 1

Encoding: Example

Input:	1	0
00	01	10
□	1	0
Output:	1	0,0

Encoding: Example

Input:	1	0	11	
	00	01	10	11
	□	1	0	11
Output:	1	0,0	01,1	

Encoding: Example

Input:	1	0	11	01	
	000	001	010	011	100
	□	1	0	11	01
Output:	1	0,0	01,1	10,1	

Encoding: Example

Input:	1	0	11	01	010	
	000	001	010	011	100	101
	□	1	0	11	01	010
Output:	1	0,0	01,1	100,0		

Encoding: Example

Input:	1	0	11	01	010	00	
	000	001	010	011	100	101	110
	□	1	0	11	01	010	00
Output:	1	0,0	01,1	100,0	010,0		

Encoding: Example

Input:	1	0	11	01	010	00	10	
	000	001	010	011	100	101	110	111
	□	1	0	11	01	010	00	10
Output:	1	0,0	01,1	100,0	010,0	001,0		

Encoding: Example

- 1 There is no apparent compression!!
- 2 Reason: The input string is too short and 0, 1 are almost equiprobable
- 3 There will be compression for long strings with unequal frequency of 0 and 1.

Decoding: Example

Input:

0

□

Output:

Decoding: Example

Input: 1

0	1
---	---

☐	1
--------------------------	---

Output: 1

Decoding: Example

Input:	1	00
00	01	10
□	1	0
Output:	1	0

Decoding: Example

Input: 1 00 011

00	01	10	11
☐	1	0	11

Output: 1 0 11

Decoding: Example

Input:	1	00	011	101
	000	001	010	011
				100
	□	1	0	11
				01
Output:	1	0	11	01

Decoding: Example

Input:	1	00	011	101	1000
	000	001	010	011	100
	1	0	11	01	010
Output:	1	0	11	01	010

Decoding: Example

Input:	1	0	011	101	1000	0100	
	000	001	010	011	100	101	110
	□	1	0	11	01	010	00
Output:	1	0	11	01	010	00	

Decoding: Example

Input:	1	0	011	101	1000	0100	0010
	000	001	010	011	100	101	110
	011	100	101	110	111		
	□	1	0	11	01	010	00
	10						
Output:	1	0	11	01	010	00	10

Few Improvements

- 1 For any string S , if both S_0 and S_1 are in the list, then S can be removed from the list.
- 2 If S and S_0 are in the list and input is S_1 , then the bit 1 need not be transmitted because it can not be 0.

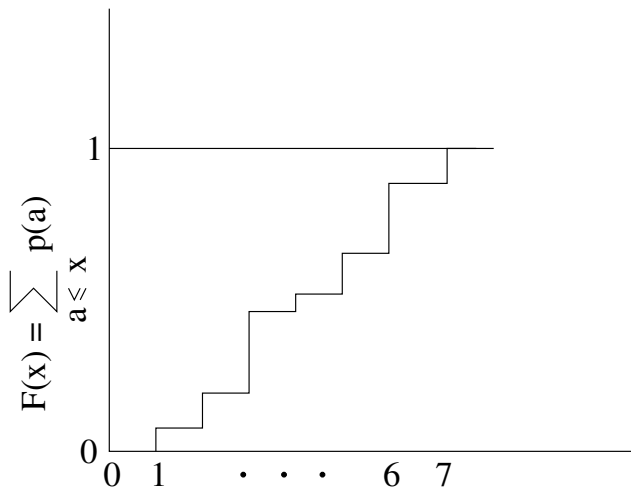
Few Improvements

- 1 For any string S , if both S_0 and S_1 are in the list, then S can be removed from the list.
- 2 If S and S_0 are in the list and input is S_1 , then the bit 1 need not be transmitted because it can not be 0.

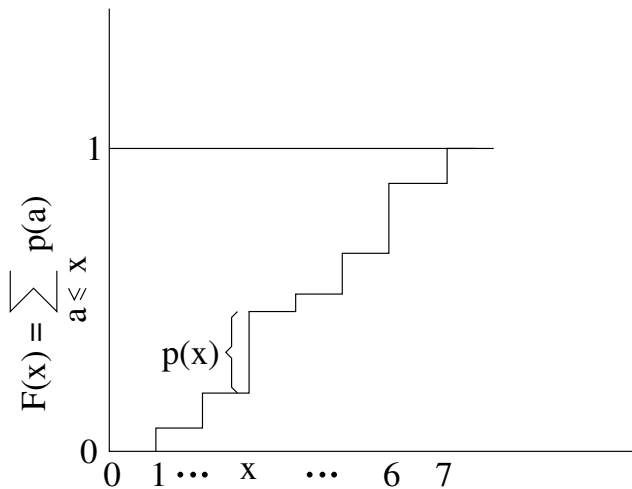
End of Lempel-Ziv coding

Shannon-Fano-Elias Code

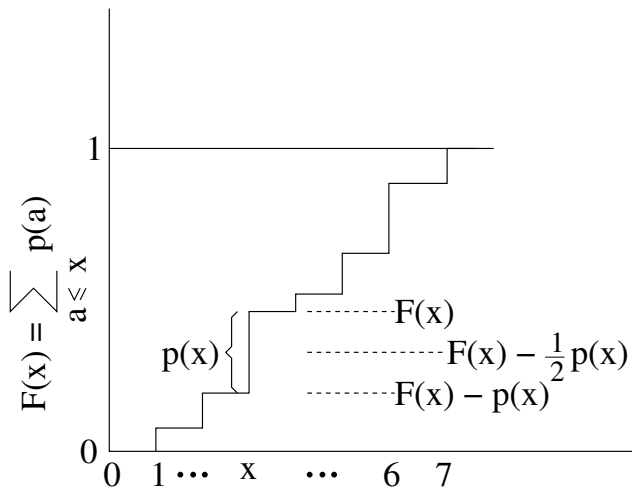
Shannon-Fano-Elias Code



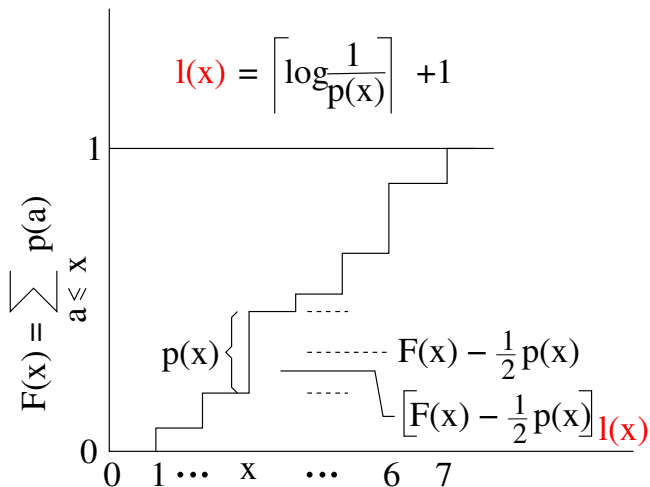
Shannon-Fano-Elias Code



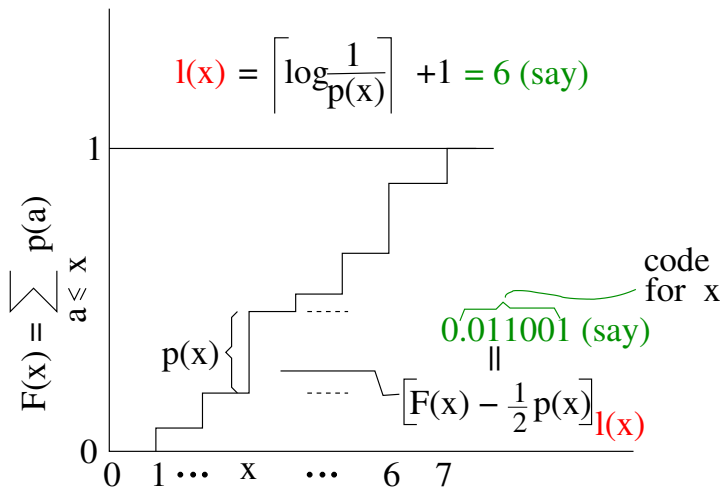
Shannon-Fano-Elias Code



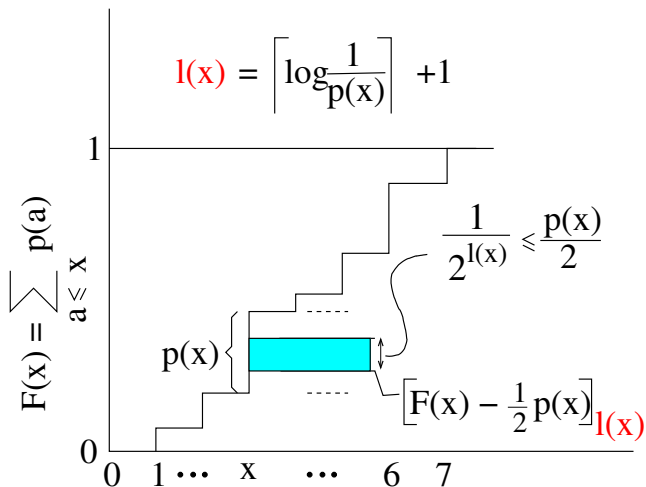
Shannon-Fano-Elias Code



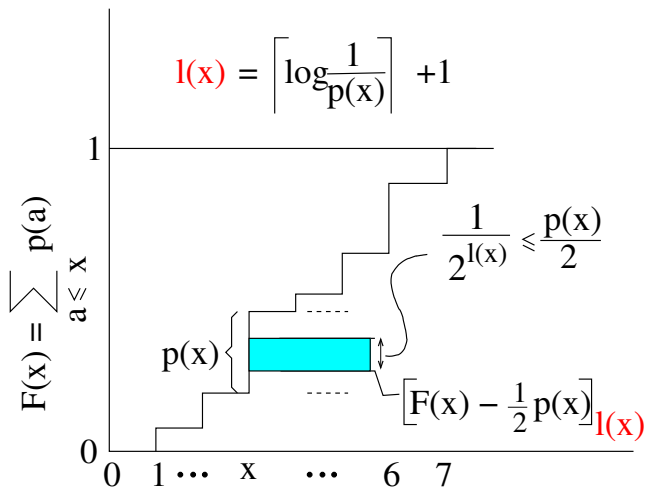
Shannon-Fano-Elias Code



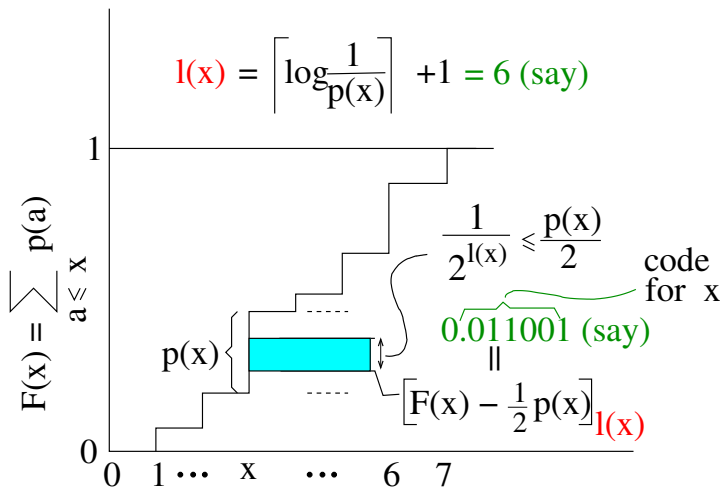
Shannon-Fano-Elias Code



Shannon-Fano-Elias Code

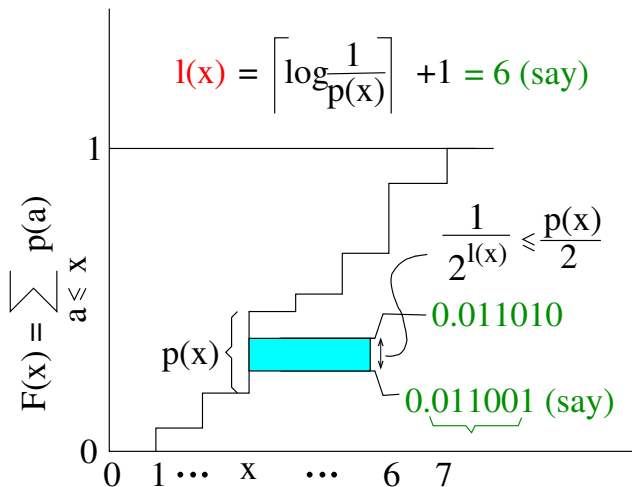


Shannon-Fano-Elias Code



Shannon-Fano-Elias Code

$$l(x) = \left\lceil \log \frac{1}{p(x)} \right\rceil + 1 = 6 \text{ (say)}$$



Performance of SFE code

$$\begin{aligned} 1. \quad L &= \sum_i P(x_i) l_i = \sum_i P(x_i) \left(\lceil \log \left(\frac{1}{P(x_i)} \right) \rceil + 1 \right) \\ &= \sum_i P(x_i) \lceil \log \left(\frac{1}{P(x_i)} \right) \rceil + 1 \\ &< \sum_i P(x_i) \log \left(\frac{1}{P(x_i)} \right) + 2 \\ &= H(X) + 2 \Rightarrow \text{suboptimal} \end{aligned}$$

2. SFE is **asymptotically optimum**

For combined coding of n symbols,

$$L = L_n/n < H(X) + 2/n \rightarrow H(X) \text{ as } n \rightarrow \infty.$$

End of Shannon-Fano-Elias Code

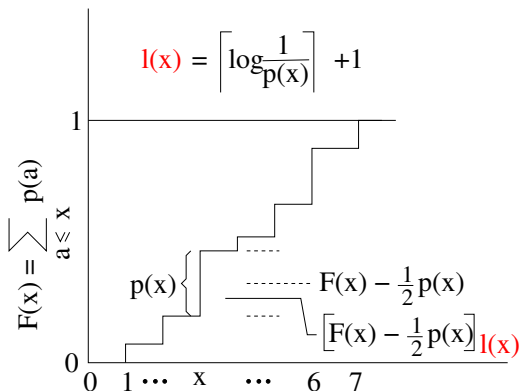
Arithmetic Code

Part 1

Arithmetic code: What is it!

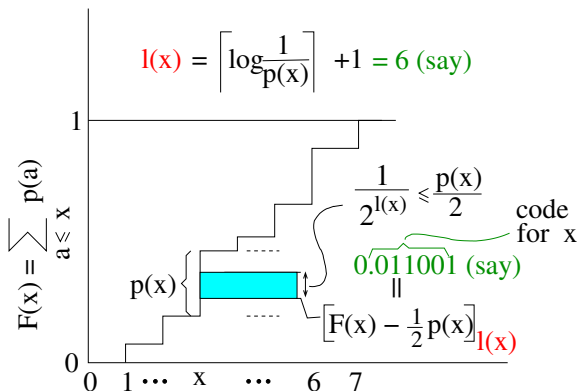
- 1 Probabilistic prediction model + SFE coding
- 2 The probabilistic model may be an i.i.d. model with a fixed distribution, a higher order model (like Markov), or an adaptive model where the distribution of the future symbols depend on the whole string seen so far.
- 3 A way of doing SFE encoding of large blocks of symbols sequentially.
- 4 Sequential encoding reduces delay.
- 5 Block length n can be increased without increasing encoding/decoding delay.

Shannon-Fano-Elias Code



- Need an ordering of the vectors.
- Need to compute $F(x)$ and $p(x)$
- If $F(x)$ and $F(x) - p(x)$ start with the same t bits out of $l(x)$ bits, then those t bits are part of the code for x .

Shannon-Fano-Elias Code



- Need an ordering of the vectors.
- Need to compute $F(x)$ and $p(x)$
- If $F(x)$ and $F(x) - p(x)$ start with the same t bits out of $l(x)$ bits, then those t bits are part of the code for x .

Block-wise SFE

Lexicography ordering of $(X_1, X_2, \dots, X_n)$

Example: The pairs $\{(x, y) : x, y \in \{1, 2, 3\}\}$ are ordered as
 $(1, 1) < (1, 2) < (1, 3) < (2, 1) < (2, 2) < (2, 3) < (3, 1) < (3, 2) < (3, 3)$.

- Similar to alphabetic ordering in a dictionary.

Computing $F_i(X_1, X_2, \dots, X_i)$ iteratively

Denote $(x_1, x_2, \dots, x_i)$ by $\mathbf{x}^{(i)}$. Then

$$\begin{aligned} F_i(\mathbf{x}^{(i)}) &= \sum_{\mathbf{y}^{(i)} \leq \mathbf{x}^{(i)}} P(\mathbf{y}^{(i)}) \\ &= \sum_{\mathbf{y}^{(i-1)} < \mathbf{x}^{(i-1)}} P(\mathbf{y}^{(i-1)}) + \sum_{y_i \leq x_i} P(\mathbf{x}^{(i-1)}, y_i) \\ &= F_{i-1}(\mathbf{x}^{(i-1)} - 1) + \sum_{y_i \leq x_i} P(\mathbf{x}^{(i-1)}) P(y_i | \mathbf{x}^{(i-1)}) \\ &= F_{i-1}(\mathbf{x}^{(i-1)} - 1) + P(\mathbf{x}^{(i-1)}) F(x_i | \mathbf{x}^{(i-1)}) \end{aligned}$$

Block-wise SFE

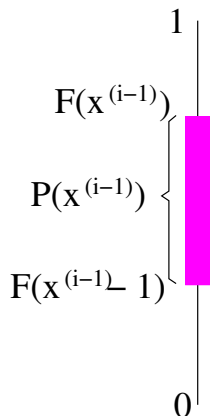
- ① Need not compute all probability masses or the complete joint cumulative distribution function.
- ② Arithmetic coding can tackle source with correlation and thus gives better compression for source with correlation.

Encoding

Computing $F_i(X_1, X_2, \dots, X_i)$ iteratively

$$F_i(\mathbf{x}^{(i)}) = F_{(i-1)}(\mathbf{x}^{(i-1)} - 1) + P(\mathbf{x}^{(i-1)})F(x_i|\mathbf{x}^{(i-1)})$$

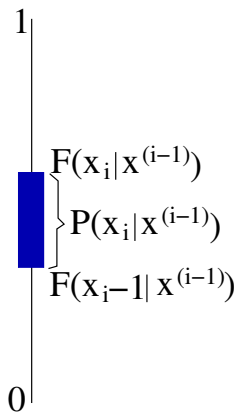
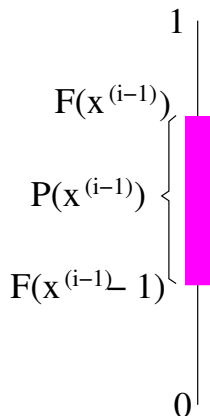
Subscript in $F(\cdot)$ is omitted for simplicity



Computing $F_i(X_1, X_2, \dots, X_i)$ iteratively

$$F_i(\mathbf{x}^{(i)}) = F_{(i-1)}(\mathbf{x}^{(i-1)} - 1) + P(\mathbf{x}^{(i-1)})F(x_i|\mathbf{x}^{(i-1)})$$

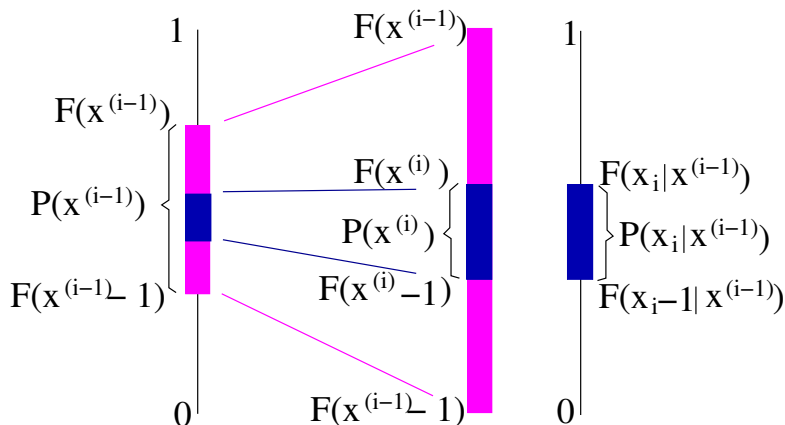
Subscript in $F(\cdot)$ is omitted for simplicity



Computing $F_i(X_1, X_2, \dots, X_i)$ iteratively

$$F_i(\mathbf{x}^{(i)}) = F_{(i-1)}(\mathbf{x}^{(i-1)} - 1) + P(\mathbf{x}^{(i-1)})F(x_i|\mathbf{x}^{(i-1)})$$

Subscript in $F(\cdot)$ is omitted for simplicity



Iterative encoding

- 1 The final encoded code-string will represent a sub-interval (a, b) in $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$. The code-string itself will be the binary representation of a .
- 2 If the binary representation of $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$ have the first t bits common, then the final code-string will start with those t bits.
- 3 As i increases, the interval $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$ will be narrower and narrower. More and more bits will be common to $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$.
- 4 So more and more bits can be encoded as i increases.

Iterative encoding

- 1 The final encoded code-string will represent a sub-interval (a, b) in $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$. The code-string itself will be the binary representation of a .
- 2 If the binary representation of $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$ have the first t bits common, then the final code-string will start with those t bits.
- 3 As i increases, the interval $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$ will be narrower and narrower. More and more bits will be common to $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$.
- 4 So more and more bits can be encoded as i increases.

Iterative encoding

- 1 The final encoded code-string will represent a sub-interval (a, b) in $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$. The code-string itself will be the binary representation of a .
- 2 If the binary representation of $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$ have the first t bits common, then the final code-string will start with those t bits.
- 3 As i increases, the interval $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$ will be narrower and narrower. More and more bits will be common to $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$.
- 4 So more and more bits can be encoded as i increases.

Iterative encoding

- 1 The final encoded code-string will represent a sub-interval (a, b) in $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$. The code-string itself will be the binary representation of a .
- 2 If the binary representation of $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$ have the first t bits common, then the final code-string will start with those t bits.
- 3 As i increases, the interval $(F_i(\mathbf{x}^{(i)} - 1), F_i(\mathbf{x}^{(i)}))$ will be narrower and narrower. More and more bits will be common to $F_i(\mathbf{x}^{(i)})$ and $F_i(\mathbf{x}^{(i)} - 1)$.
- 4 So more and more bits can be encoded as i increases.

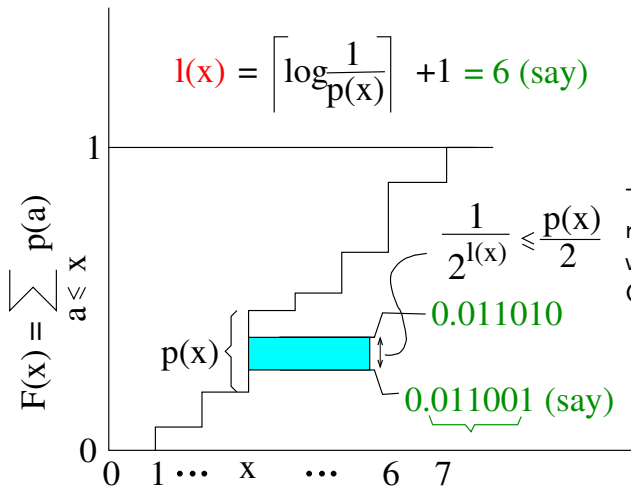
End of Arithmetic Code - Part 1

Arithmetic Code

Part 2

Shannon-Fano-Elias Code

$$l(x) = \left\lceil \log \frac{1}{p(x)} \right\rceil + 1 = 6 \text{ (say)}$$

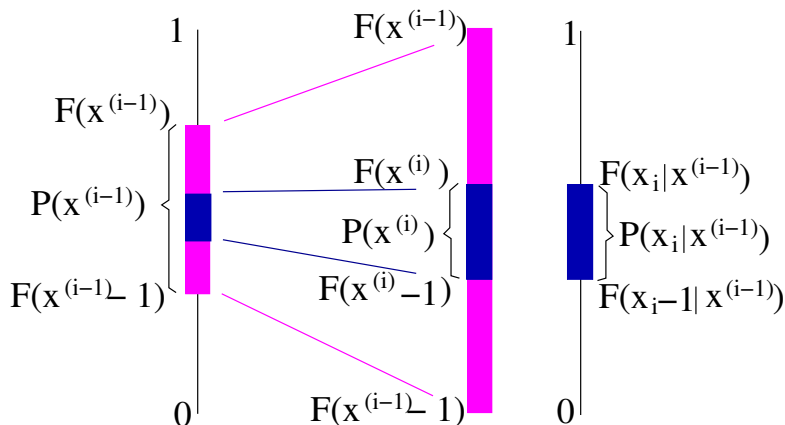


The sequence should represent an interval within the “jump” in the CDF

Computing $F_i(X_1, X_2, \dots, X_i)$ iteratively

$$F_i(\mathbf{x}^{(i)}) = F_{(i-1)}(\mathbf{x}^{(i-1)} - 1) + P(\mathbf{x}^{(i-1)})F(x_i|\mathbf{x}^{(i-1)})$$

Subscript in $F(\cdot)$ is omitted for simplicity



Arithmetic Code

Encoding a sequence of unknown statistics

Estimating $P(x_i|\mathbf{x}^{(i-1)})$

Special Cases

① **I.I.D. source:** $P(x_i|\mathbf{x}^{(i-1)}) = P(x_i)$

② **Markov source:** $P(x_i|\mathbf{x}^{(i-1)}) = P(x_i|x_{i-1})$

Computing $P(x_i|\mathbf{x}^{(i-1)})$

Models for unknown source statistics

1 Laplace model:

$$P(X_i = a|\mathbf{x}^{(i-1)}) = \frac{F_a + 1}{\sum_b (F_b + 1)}$$

where F_a is the number of a in $\mathbf{x}^{(i-1)}$.

2 Dirichlet model:

$$P(X_i = a|\mathbf{x}^{(i-1)}) = \frac{F_a + \alpha}{\sum_b (F_b + \alpha)}$$

A smaller α tries to learn the statistics faster.

Computing $P(x_i|\mathbf{x}^{(i-1)})$

Models for unknown source statistics

1 Laplace model:

$$P(X_i = a|\mathbf{x}^{(i-1)}) = \frac{F_a + 1}{\sum_b (F_b + 1)}$$

where F_a is the number of a in $\mathbf{x}^{(i-1)}$.

2 Dirichlet model:

$$P(X_i = a|\mathbf{x}^{(i-1)}) = \frac{F_a + \alpha}{\sum_b (F_b + \alpha)}$$

A smaller α tries to learn the statistics faster.

Example: File Compression

Binary file $\Rightarrow$ three source characters:

- ① 0: will be denoted by **a** to distinguish from the encoded bit 0.
- ② 1: will be denoted by **b** to distinguish from the encoded bit 1.
- ③ **end-of-file** character denoted by **□**.

Probability model

$$\textcircled{1} \quad P(\square | \mathbf{x}^{(i-1)}) = 0.15$$

$$\textcircled{2} \quad P(x_i = a | \mathbf{x}^{(i-1)}) = 0.85 \times \frac{F_a + 1}{F_a + F_b + 2}$$

$$\textcircled{3} \quad P(x_i = b | \mathbf{x}^{(i-1)}) = 0.85 \times \frac{F_b + 1}{F_a + F_b + 2}$$

We take a short file **bbba**□ for illustration.

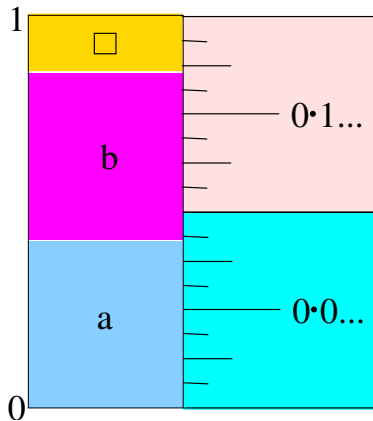
Arithmetic encoding: Example

We need to compute the following probabilities:

Context $\mathbf{x}^{(i-1)}$	Conditional Probabilities		
	$P(a \mathbf{x}^{(i-1)})$	$P(b \mathbf{x}^{(i-1)})$	$P(\square \mathbf{x}^{(i-1)})$
	0.425	0.425	0.15
b	0.28	0.57	0.15
bb	0.21	0.64	0.15
bbb	0.17	0.64	0.15
bbba	0.28	0.57	0.15

File compression

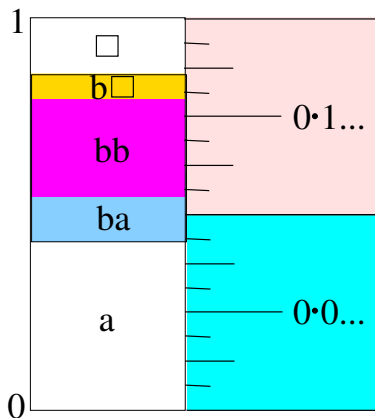
Input: **b**



Output:

File compression

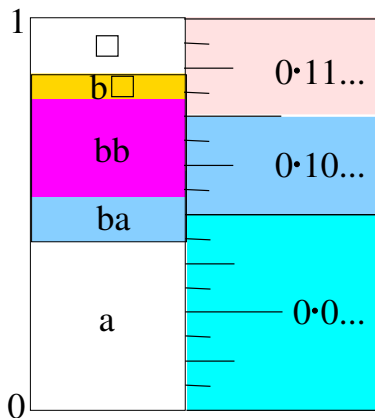
Input: b b



Output: 1

File compression

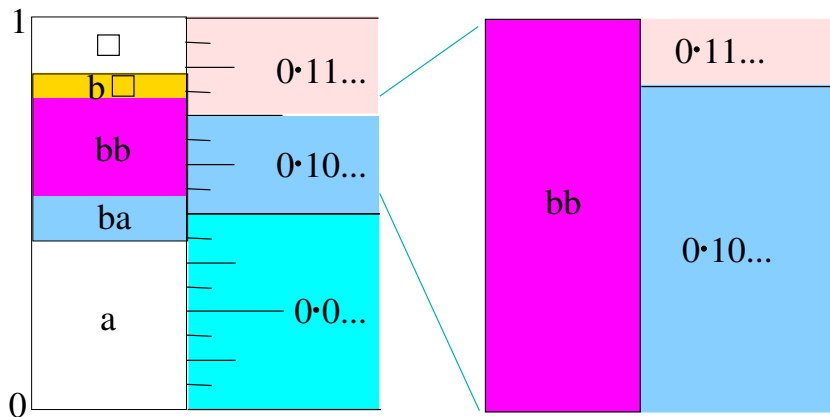
Input: $b \quad b$



Output: 1

File compression

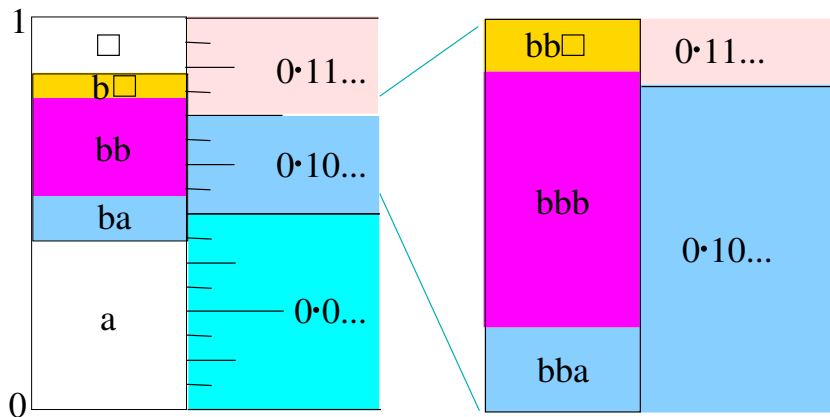
Input: **b** **b**



Output: **1**

File compression

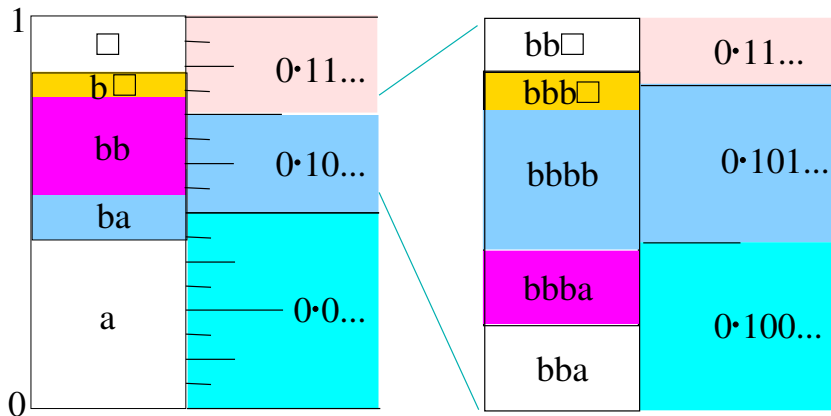
Input: b b b



Output: 1

File compression

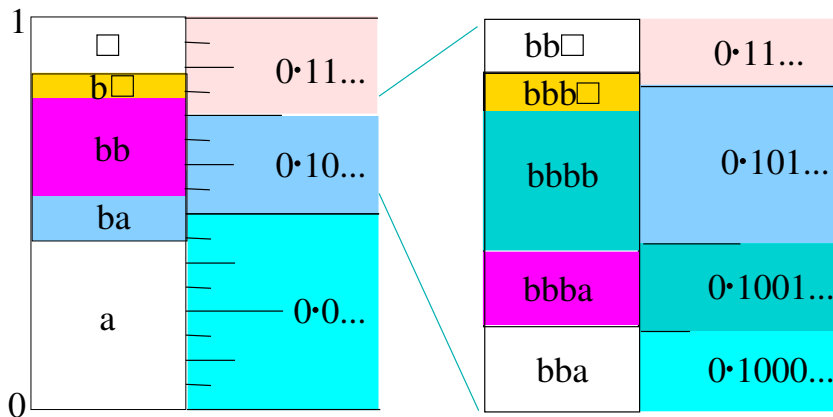
Input: b b b a



Output: 1

File compression

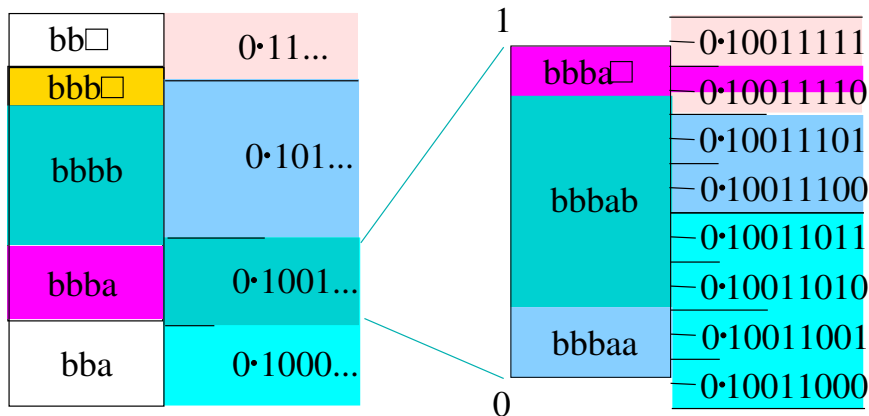
Input: b b b a



Output: 1 001

File compression

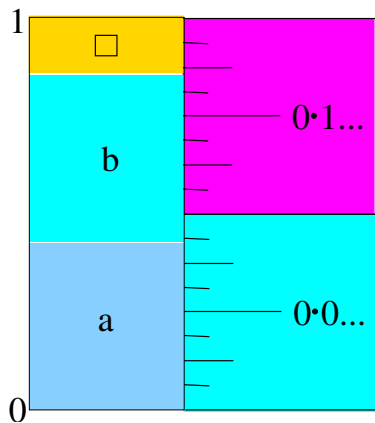
Input: b b b a □



Output: 1 0 0 1 11101

File compression: Decoding

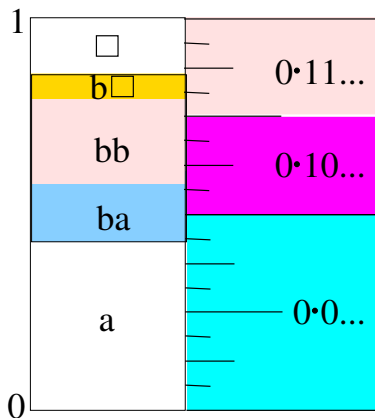
Input: 1



Output:

File compression: Decoding

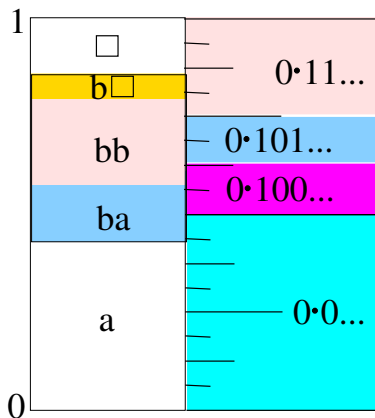
Input: 1 0



Output: b

File compression: Decoding

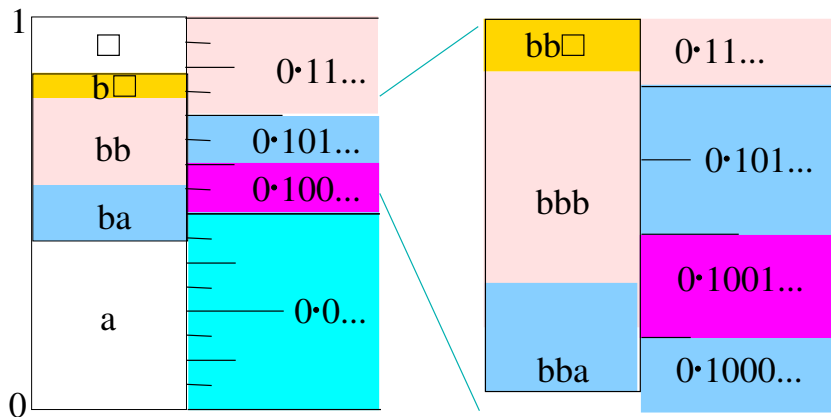
Input: 1 0 0



Output: b

File compression: Decoding

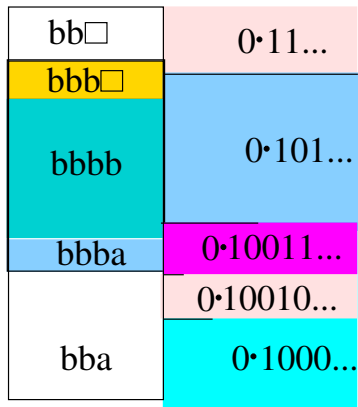
Input: 1 0 0 1



Output: b b

File compression: Decoding

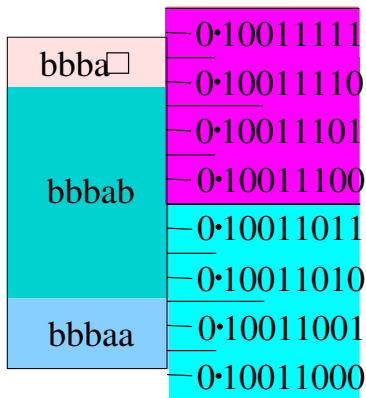
Input: 1 0 0 1 1



Output: b b

File compression: Decoding

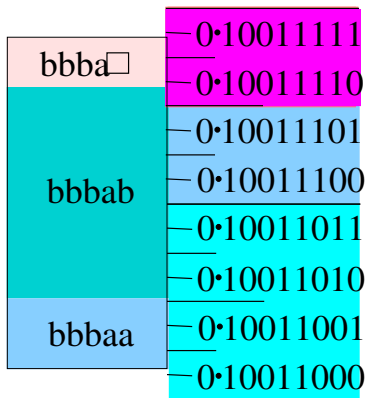
Input: 1 0 0 1 1 1



Output: b b b

File compression: Decoding

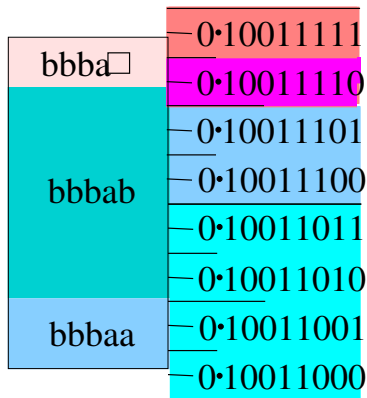
Input: 1 0 0 1 1 1 1



Output: b b b

File compression: Decoding

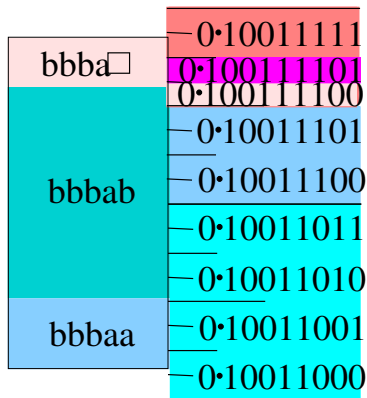
Input: 1 0 0 1 1 1 1 0



Output: b b b a

File compression: Decoding

Input: 1 0 0 1 1 1 1 0 1



Output: b b b a □

End of Arithmetic Code - Part 2